

Drift-Sense

DRAM-SEM Reference Localizer

A Complete Guide to Setup, Training, Inference, and Evaluation

A deep-learning replacement for classical ZNCC template matching, locating a small Reference SEM image inside a larger Search SEM image at 10× zoom difference — using dense correlation and a global-receptive-field context head to resolve the periodic-lattice ambiguity that defeats purely local matching on DRAM patterns.

Package: `puthere/` — Checkpoint: `m3_hn_r24/best.pt`

Table of Contents

1.	Overview	3
2.	Package Contents	4
3.	Setup and Installation	5
4.	Quick Start — Running Predictions	6
5.	Training Your Own Model	8
6.	Evaluating Against Baselines	10
7.	About the Provided Checkpoint	11
8.	Calibration and Diagnostic Scripts	12
9.	Generating Your Own Dataset	14
10.	Architecture Walkthrough	15
11.	Troubleshooting	17
12.	Further Reading	18

1. Overview

This package finds where a small **Reference** SEM (scanning electron microscope) image is located inside a larger **Search** SEM image of the same sample region, imaged at 10× lower resolution (1 real-world nanometre spans 10× more Search pixels than Reference pixels). The output is a predicted centre coordinate (x , y) in Search-image pixel space, plus a confidence score.

The classical approach — normalized cross-correlation (ZNCC) sliding-window template matching — struggles specifically on DRAM patterns, because DRAM word lines and bit lines repeat with near-perfect periodicity every few pixels. A Reference crop can look almost pixel-identical to several different lattice repeats elsewhere in the Search image, and a purely local matcher has no way to tell them apart.

The core idea

This model resolves that ambiguity with two ingredients working together:

- **Dense correlation** — a Siamese (shared-weight) encoder produces feature maps for both images, and every Reference-feature cell is cross-correlated against every Search-feature position (channel-preserving, not collapsed to a single similarity score) — similar to classical template matching, but with learned features instead of raw pixels.
- **A global-receptive-field context head** — a stack of 9 dilated convolutions (dilations 1, 2, 4, 8, 16, 32, 64, 1, 1) processes the correlation volume with a receptive field wide enough to see the *entire* correlation map at once. This is the actual disambiguator: it can use far-away context — die/mat boundaries, region-specific imaging noise — to decide which of several locally-identical matches is the real one, something a local encoder alone cannot do.

The result is decoded into a heatmap peak plus a sub-pixel offset correction, then a small number of deliberate, tested rules (non-maximum suppression, a closest-to-image-centre tie-break for genuinely ambiguous peaks, and a peak-margin-based confidence score) turn that into the final (x , y , confidence) answer.

NOTE — Why this matters for the guide below

Every script in this package is a piece of this pipeline: `train.py` trains it, `predict.py` runs it on new images, and the calibration/ablation scripts each validate one specific design decision (the tie-break threshold, whether the model relies on real signal vs. synthetic noise fingerprints, whether confidence is actually informative). Section 10 walks through the architecture in more depth for anyone modifying the code.

2. Package Contents

This folder is **fully self-contained** — it does not depend on the original development repository. Everything needed to install, run, retrain, and evaluate the model is here.

```
src/
  pipeline.py, sem_imaging.py, presets.py, structural_defects.py      # synthetic SEM canvas generator (shared infra)
  patterns/                                                         # DRAM / FinFET / zone-routing pattern generators
  localizer/                                                         # the model itself
    geometry.py                                                     # coordinate-mapping constants
    config.py                                                       # LocalizerConfig -- every tunable in one place
    data.py                                                         # on-the-fly training/val/test data generation
    encoder.py                                                      # Siamese ResNet-18 (destrided, no pretraining)
    correlation.py                                                  # dense depthwise cross-correlation
    context_head.py                                                 # 9-layer dilated conv stack
    targets.py                                                      # Gaussian heatmap + offset targets
    losses.py                                                       # focal heatmap, offset L1, hard-negative
    decode.py                                                       # heatmap -> (x, y, confidence)
    model.py                                                        # assembles everything
    metrics.py                                                      # acc@k, AP, PR curve

baseline_solution/                                                  # classical ZNCC (B0) and grid-search (B1) baselines

scripts/
  train.py                                                          # train a model
  predict.py                                                        # run inference on one reference/search pair
  evaluate_localizer.py                                             # compare B0 / B1 / B3 side by side
  calibrate_tie_ratio.py                                           # calibrate decode's tie-break threshold
  benchmark_runtime.py                                             # per-stage FP32/FP16 timing
  ablation_jitter.py                                               # A1: synthetic-noise-fingerprint dependence
  ablation_negative.py                                             # A2: confidence signal on absent references

checkpoints/m3_hn_r24/best.pt                                       # a trained checkpoint (see Section 7)
tests/                                                             # pytest suite
docs/superpowers/                                                  # design spec, build plan, milestone results
generate_mxn_dram_dataset.py                                       # standalone dataset generation script
requirements.txt, pytest.ini
```

3. Setup and Installation

3.1 Create an environment and install dependencies

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

WARNING — The torch/torchvision pin needs an extra index URL

`requirements.txt` pins `torch==2.13.0+cu130` and `torchvision==0.28.0+cu130`. The `+cu130` suffix is a PEP 440 *local version label* — those specific wheels are only published on PyTorch's own index, not plain PyPI, so a bare `pip install -r requirements.txt` will fail to resolve them. Install explicitly instead:

```
pip install torch==2.13.0+cu130 torchvision==0.28.0+cu130 \
  --extra-index-url https://download.pytorch.org/whl/cu130
```

If you're on a different CUDA version, or CPU-only, install whatever build matches your machine — the code is not tied to that specific build; it's simply what this package was developed and validated against. CPU-only is fine for running `predict.py` on a single image pair; training is impractically slow without a GPU.

3.2 Verify the install

```
python -m pytest tests/ -q -m "not slow"
```

Expected: 96 passed, 2 deselected. The two deselected tests are marked `@pytest.mark.slow` (a real receptive-field measurement and a short live-training integration test) — include them with `-m slow`, or drop the `-m` filter entirely to run the full suite (about 40 seconds instead of 35).

All commands in the rest of this guide assume you're running from this package's root directory, with the virtual environment active.

4. Quick Start — Running Predictions

The fastest way to see the model work is `scripts/predict.py`, which loads a checkpoint and runs it on one reference/search image pair.

```
python scripts/predict.py \
  --checkpoint checkpoints/m3_hn_r24/best.pt \
  --reference path/to/reference.png \
  --search path/to/search.png \
  --verbose
```

Output:

```
predicted_x=484.77 predicted_y=898.32 confidence=0.0745
```

Omit `--verbose` for a compact, machine-readable `x,y,confidence` line on its own — useful for piping into another script.

4.1 Input requirements

The script validates the search image's geometry before running the model, and raises a clear error rather than letting a wrong-size input fail deep inside the decode logic:

- **Search image** must be exactly 1000×1000 pixels, grayscale. A different size raises a `ValueError` naming both the expected and actual size.
- **Reference image** is automatically resized to 100×100 pixels. This assumes the reference is already at the correct physical scale (imaged at 10× the Search image's resolution over the same real-world area) — a reference at a genuinely different physical scale will be silently resized to the wrong extent and produce a confidently wrong answer, since pixel data alone can't reveal the mismatch.

4.2 Interpreting confidence

Confidence is the **margin** between the chosen heatmap peak and the next-strongest peak elsewhere on the map — not a calibrated probability and not the raw peak height. This is deliberate: the classical ZNCC baseline scored a misleadingly high 0.9+ average similarity even on a test set where it picked the wrong location about 35% of the time, which is exactly the failure mode a margin-based score is designed to avoid (see Section 8.4 for the diagnostic that validates this).

- Use confidence to **rank** multiple predictions against each other, not as an absolute pass/fail threshold.
- Confidence **can be slightly negative**. This happens specifically when the decode logic's closest-to-image-centre tie-break overrides raw peak ranking (choosing a near-tied peak that isn't the single highest-scoring one) — it's expected behaviour, an honest signal that the match was ambiguous, not an error to guard against.

5. Training Your Own Model

```
python scripts/train.py --run-name my_run --max-steps 40000
```

Data is generated on-the-fly from a seed — there is no dataset directory to point at. Training, validation, and test splits are canvas-disjoint by seed range (different seed ranges generate structurally different synthetic canvases), so validation numbers are never contaminated by near-duplicate crops of a training canvas.

5.1 Key flags

Flag	Default	Meaning
--run-name	required	Checkpoints are written to checkpoints/<run-name>/best.pt.
--max-steps	40000	The real training budget. Expect several hours on a single consumer GPU at this size.
--no-context	off	Trains the context-free (“B2”) ablation instead of the real model — useful only for comparing against the full architecture, not for a model you intend to use.
--jitter-profile	normal	normal / zero / shifted — controls the synthetic imaging-noise profile used during data generation. Only relevant for the A1 ablation in Section 8.3; leave at the default otherwise.
--lambda-hn	0.0	Hard-negative loss weight. The <i>provided</i> checkpoint was trained with 0.5 — pass --lambda-hn 0.5 to replicate that.
--hn-radius	config (24)	Hard-negative exclusion radius, in correlation cells. 24 was selected by a 4-way sweep (6 / 12 / 24 / 48) during development — override only if you have reason to.
--lr, --batch-size	config	Override LocalizerConfig's calibrated defaults if omitted, the config's own values are used (never a hidden hardcoded default competing with them).
--val-every, --val-batches	1000 / 40	Validation cadence and how many batches each validation pass evaluates.
--num-workers	6	DataLoader worker count for on-the-fly generation.

5.2 Reading the training log

Every validation step prints two lines:

```
[val] acc@50px 0.860  acc@5px 0.840  median 1.0px  AP 0.846  
[val] std_x 258.40px  std_y 246.12px  distinct_preds 64/64
```

The second line is a **collapse monitor**. If std_x / std_y shrink toward 0, or distinct_preds shrinks toward 1, the model is predicting the same point regardless of input — a real failure mode this architecture hit several times during development (see the design/build-plan docs in Section 12 for the full post-mortems). A healthy model keeps both standard deviations large and distinct_preds near the total sample count.

NOTE — Why this monitor exists

Every serious bug found during this project's development was invisible to shape-only unit tests and only surfaced by actually running training or inference and noticing an oddly round number — e.g. every single prediction landing on exactly the same pixel. This log line makes that failure mode visible immediately, in every run, without extra tooling.

6. Evaluating Against Baselines

```
python scripts/evaluate_localizer.py \  
    --checkpoint checkpoints/m3_hn_r24/best.pt --n-samples 500
```

Runs three methods on the same held-out, canvas-disjoint test samples and prints an accuracy table:

Method	What it is
B0	Classical ZNCC sliding-window template matching — the pre-deep-learning baseline this project set out to beat.
B1	A 10×10 grid-search baseline with an analytically-derived accuracy ceiling, used to sanity-check that acc@k numbers are self-consistent.
B3	This model (dense correlation + context head). Pass <code>--no-context</code> to evaluate a B2 (context-free) checkpoint instead.

Results are written to `eval_results/comparison.json` for further analysis.

7. About the Provided Checkpoint

WARNING — Read this before trusting the shipped checkpoint's numbers

checkpoints/m3_hn_r24/best.pt was trained for only **1000 steps**, not the 40000-step production budget described in Section 5. This was a deliberate development-time choice: validate that the full pipeline (data generation, loss, training loop, decoding) works correctly end-to-end and that loss decreases sensibly, without spending hours of GPU time per experiment.

At 1000 steps it already reaches **acc@50px** $\approx$ **0.86–0.88** on held-out test samples (the full milestone table is in docs/superpowers/plans/results-m1-m3.md) — good enough to demo and sanity-check the pipeline end-to-end, and the baked-in `hard_negative_radius_cells=24` is a real, validated result from a 4-way radius sweep (6 / 12 / 24 / 48), not a guess.

But it has **not** been run at the real training budget. Don't treat its absolute accuracy as representative of what the architecture can achieve — retrain with `--max-steps 40000` (or higher) for a production-quality model:

```
python scripts/train.py --run-name production --max-steps 40000 --lambda-hn 0.5
```

8. Calibration and Diagnostic Scripts

8.1 Tie-break threshold calibration

`decode()`'s tie-break threshold (`peak_tie_ratio`) is calibrated on validation data, never picked arbitrarily. Re-run after retraining rather than reusing the shipped value (0.98):

```
python scripts/calibrate_tie_ratio.py --checkpoint checkpoints/m3_hn_r24/best.pt
```

Prints a sensitivity curve over $\tau \in [0.80, 1.00]$ and the selected value. Update `LocalizerConfig.peak_tie_ratio` in `src/localizer/config.py` by hand to bake a new value into future runs.

8.2 Runtime benchmarking

```
python scripts/benchmark_runtime.py
```

Per-stage FP32/FP16 timing (encoder, correlation, context head, decode), measured with CUDA events, compared against the classical ZNCC baseline's CPU runtime.

8.3 A1: jitter-fingerprint dependence ablation

Tests whether the model relies on generator-specific synthetic noise fingerprints rather than genuine aperiodicity — a real overfitting risk that would not transfer to actual SEM data. Requires a second checkpoint trained on the degenerate zero-jitter profile:

```
python scripts/train.py --run-name zero_jitter_model \
  --jitter-profile zero --max-steps 40000
python scripts/ablation_jitter.py \
  --checkpoint checkpoints/m3_hn_r24/best.pt \
  --checkpoint-zero checkpoints/zero_jitter_model/best.pt
```

Prints a 2×3 train-profile $\times$ test-profile accuracy matrix:

- A large **normal**→**zero** accuracy drop is **expected** — the zero-jitter lattice is exactly periodic, so it's genuinely, provably ambiguous by construction, not a model failure.
- A large **normal**→**shifted** drop would be the actual red flag — it would mean the model overfit to one specific noise distribution rather than aperiodicity in general, and would not transfer to real SEM data.

8.4 A2: negative-reference ablation (diagnostic — gates nothing)

Pairs each Search image with a Reference cut from a genuinely unrelated canvas, so the target is truly absent, and measures whether confidence actually separates “present” from “absent” cases (something the classical ZNCC baseline is bad at — see Section 4.2):

```
python scripts/ablation_negative.py --checkpoint checkpoints/m3_hn_r24/best.pt
```

Prints an AUROC. Well above 0.5 means the margin-based confidence is genuinely informative; near 0.5 means it isn't and the confidence definition needs revisiting. This ablation is purely diagnostic — the deployed task always guarantees the reference is present, so it gates no milestone.

9. Generating Your Own Dataset

```
python generate_mxn_dram_dataset.py --help
```

Generates paired Reference/Search images with an $m \times n$ multi-region DRAM die/array layout, plus a `manifest.csv` recording ground-truth coordinates for each pair.

NOTE — You usually don't need this

`train.py`, `evaluate_localizer.py`, and the ablation scripts all generate data on-the-fly from a random seed — that's why none of them take a `--data-dir` flag. This script exists for cases where you want a *persisted* dataset for external tooling (a demo, a different framework, manual inspection), not because the pipeline itself needs one.

10. Architecture Walkthrough

This section is for anyone reading or modifying the code in `src/localizer/`. It is not required to just run the scripts above.

10.1 The pipeline, stage by stage

Stage	Module	What it does
1. Encode	<code>encoder.py</code>	A Siamese (shared-weight) ResNet-18, destride so its output stride is 4 instead of the usual 32, with a 1-channel grayscale stem and no pretrained weights (training data is unlimited, generated on-the-fly, so there's no benefit to ImageNet features). Both Reference and Search images go through the same encoder.
2. Correlate	<code>correlation.py</code>	Dense depthwise cross-correlation between the Search feature map and the Reference feature map at every valid offset — channel-preserving (128 channels kept, not collapsed to a single similarity score per position), giving the context head real per-channel signal to work with.
3. Contextualize	<code>context_head.py</code>	9 stacked dilated convolutions (dilations 1, 2, 4, 8, 16, 32, 64, 1, 1), with a theoretical receptive field of 259 cells $\geq$ the full 226-cell correlation map — verified empirically, not just by the dilation-rate arithmetic, via a gradient-based effective-receptive-field measurement (see the test suite). Outputs a heatmap logit and a 2-channel sub-cell offset field.
4. Target	<code>targets.py</code>	During training only: converts a ground-truth (x, y) into a soft Gaussian heatmap target plus a per-cell offset target, so the loss has a smooth target to fit rather than a single hard one-hot cell.
5. Loss	<code>losses.py</code>	Focal loss on the heatmap (CenterNet-style, penalty-reduced near the true peak) + L1 loss on the sub-cell offset + an optional hard-negative margin loss that explicitly penalizes high-confidence false peaks near — but not at — the true location.
6. Decode	<code>decode.py</code>	Non-maximum suppression, then among genuinely tied top peaks (within a calibrated ratio of the best score) picks the one closest to the image centre — a decode-time-only rule that never appears in any loss term, since ground-truth centres are uniformly distributed and a learned centre prior would be actively harmful. Confidence is the margin between the chosen peak and the next-strongest peak elsewhere on the map.

10.2 Coordinate geometry

All coordinate mapping goes through `src/localizer/geometry.py`, the single source of truth — nowhere else in the codebase re-derives these numbers:

```
SCALE      = 10      # reference nm/px : search nm/px (the 10x zoom difference)
STRIDE     = 4        # encoder output stride
REF_PX     = 1000     # reference native size, pixels
SEARCH_PX  = 1000     # search native size, pixels
REF_DS_PX  = 100      # reference resized to search-image scale (REF_PX / SCALE)
HALF       = 50       # template half-size (REF_DS_PX / 2)
REF_FEAT   = 25       # reference feature-map size (REF_DS_PX / STRIDE)
```

```

SEARCH_FEAT = 250          # search feature-map size (SEARCH_PX / STRIDE)
CORR         = 226          # valid-mode correlation map size
                        # (SEARCH_FEAT - REF_FEAT + 1)

# cell index -> pixel: x = STRIDE * cell + HALF + align_offset
# align_offset is measured empirically (encoder.calibrate_alignment), not
# assumed -- the true stem-padding alignment can't be derived analytically.

```

10.3 Every tunable, in one place

`src/localizer/config.py`'s `LocalizerConfig` dataclass holds every hyperparameter — nothing is a magic number buried elsewhere in the code. The values below are the current, calibrated defaults (what the shipped checkpoint was trained/decoded with, except `lambda_hard_negative` and `batch_size`, which reflect the code's committed defaults rather than the specific training run):

Field	Value	Notes
<code>encoder_stride</code>	4	Fixed by an aliasing argument in the design spec.
<code>corr_channels</code>	128	Correlation channel depth.
<code>context_channels</code>	64	Context head hidden width.
<code>context_dilations</code>	(1,2,4,8,16,32,64,1,1)	Gives a 259-cell theoretical RF.
<code>heatmap_sigma_cells</code>	2.0	Swept over {1, 2, 3} during development.
<code>lambda_offset</code>	1.0	Offset-loss weight.
<code>lambda_hard_negative</code>	0.5	0 disables it (the "M2" ablation state).
<code>hard_negative_radius_cells</code>	24	Selected by a sweep over {6, 12, 24, 48}.
<code>focal_alpha / focal_beta</code>	2.0 / 4.0	CenterNet-style focal loss exponents.
<code>peak_tie_ratio</code>	0.98	Calibrated on validation, swept over [0.80, 1.00].
<code>nms_kernel</code>	3	Non-maximum-suppression window size.
<code>lr / weight_decay</code>	1e-3 / 1e-4	AdamW optimizer settings.
<code>batch_size / max_steps</code>	8 / 40000	Training loop defaults.
<code>crops_per_canvas</code>	100	Reference/Search crops drawn per generated canvas.

11. Troubleshooting

Symptom	Likely cause / fix
<code>pip install -r requirements.txt</code> fails to find torch	The +cu130 version needs <code>--extra-index-url https://download.pytorch.org/whl/cu130</code> (Section 3.1), or install a build matching your own CUDA version / CPU-only instead.
<code>predict.py</code> raises a <code>ValueError</code> about image size	The search image must be exactly 1000×1000 px. Resize it, or double-check you didn't pass the reference and search paths in swapped order.
<code>predict.py</code> gives a confidently wrong answer	Check the reference image is genuinely at the same physical scale the model expects (10× zoomed-in relative to the search image) — the resize to 100×100 px assumes this and cannot detect a mismatch from pixel data alone.
Training loss prints nan and never recovers	This was a real bug found and fixed during development (an fp16-AMP precision issue in the focal loss). If you're running the exact code in this package, it's already fixed — if you've modified <code>losses.py</code> , check that loss computations that involve <code>log()</code> near 0 or 1 are cast to fp32 before the log, regardless of the ambient autocast dtype.
Validation shows <code>distinct_preds</code> stuck near 1	The model is collapsing to a constant prediction — a real failure mode this architecture hit multiple times during development (see Section 5.2's collapse monitor, and the design/build-plan docs in Section 12 for the specific root-caused bugs and fixes). Worth checking against the checkpoint provided in this package as a known-good baseline.
Training uses far more RAM than expected	<code>LocalizerDataset</code> uses a shuffle buffer (default 256 samples per worker) to give real batch diversity, at the cost of holding that many full-resolution search images in memory per worker — roughly 1 GB/worker at default settings. Reduce <code>--num-workers</code> , or construct <code>LocalizerDataset</code> with a smaller <code>shuffle_buffer_size</code> directly if modifying the code.
GPU not being used	Confirm <code>torch.cuda.is_available()</code> returns <code>True</code> in your environment; all scripts auto-detect CUDA and fall back to CPU silently.

12. Further Reading

Everything referenced below ships inside this package.

- `docs/superpowers/specs/2026-08-08-dram-sem-reference-localization-design.md` — the architecture design spec: why dense correlation plus a context head, why a Gaussian heatmap target, why the loss is structured the way it is.
- `docs/superpowers/plans/2026-08-08-dram-sem-localizer.md` — the full build plan, including the exact coordinate-mapping derivation, every configurable hyperparameter, and the milestone accuracy gates.
- `docs/superpowers/plans/results-m1-m3.md` — the actual milestone training results, with the scaled-down-training-run caveat (Section 7 of this guide) spelled out in full.
- **Module docstrings and inline comments** in `src/localizer/decode.py` and `src/localizer/model.py` in particular explain several non-obvious design decisions worth reading before modifying either file: why confidence can go negative, why sigmoid must always be applied unconditionally to the heatmap logits, and why the centre-tiebreak rule must never appear in any loss term.

Generated for the Drift-Sense DRAM-SEM Reference Localizer package.